



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 08

NOMBRE COMPLETO: Tavera Castillo David Emmanuel

N° de Cuenta: 320054831

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 10/23/2025

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

El primer ejercicio nos pide agregar un spotlight que parta del cofre del coche de la practica anterior y que al abrir y cerrar dicho cofre se ilumine en la dirección correcta.

Primero crearemos un spot light de color naranja.

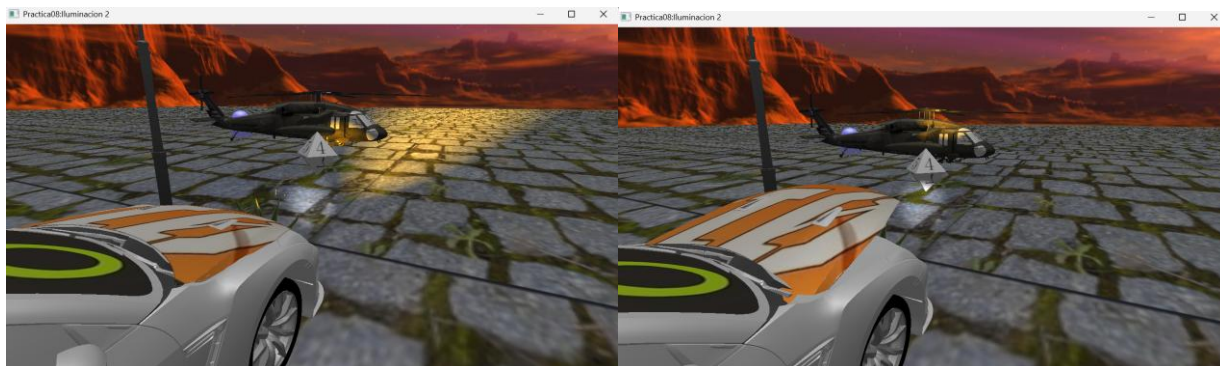
```
spotLights[2] = Spotlight(1.0f, 0.647f, 0.0f,
    1.0f, 2.0f,
    0.0f, 0.0f, 0.0f, //Posicion
    0.0f, 0.0f, 1.0f, //Direccion
    1.0f, 0.0f, 0.0f,
    15.0f);
spotLightCount++;
```

Después ligamos la posición de la luz con el cofre, para ser mas exactos con la punta del cofre.

```
//Cofre
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 2.3198f, 9.0361f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(1.0f, 0.0f, 0.0f));
modelaux = model;
pivotWorld = glm::vec3(cochePos.x, 3.6599f+cochePos.y, -10.482f+cochePos.z);
tipLocal = glm::vec3(0.0f, -0.8839f, 4.604f);
tipWorld = pivotWorld + glm::vec3(glm::rotate(glm::mat4(1.0f), glm::radians(mainWindow.getarticulacion2()), glm::vec3(1.0f, 0.0f, 0.0f)) * glm::vec4(tipLocal, 1.0f));
spotLights[2].setFlash(tipWorld, glm::vec3(0.0f, -sin(glm::radians(mainWindow.getarticulacion2())), cos(glm::radians(mainWindow.getarticulacion2()))));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Cofre.RenderModel();
```

Como la punta del cofre depende de la rotación de este debemos de realizar ciertos cálculos que mueve nuestra luz con la rotación del cofre y además rote la luz para que tenga el efecto de que siempre apunta hacia enfrente con respecto del cofre.

Para poder apreciar los resultados de nuestras líneas de código instanciamos el helicóptero de prácticas pasadas, en este vamos a observar el comportamiento del spotlight.

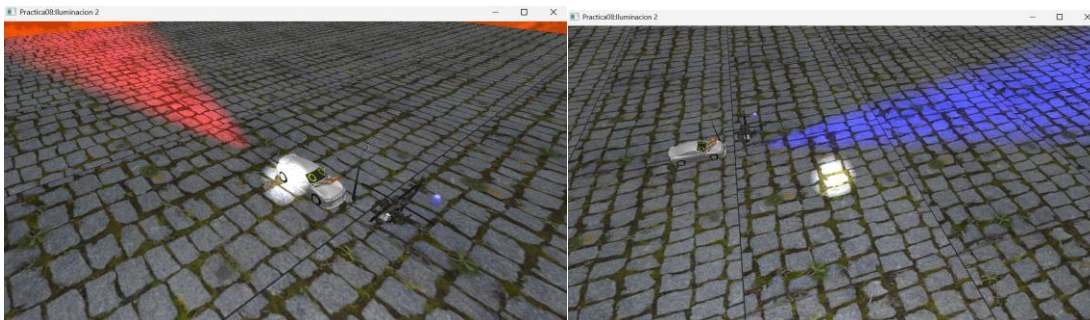


Si el coche se mueve también lo hará dicha luz.

El segundo ejercicio consta de agregar una nueva spotlight de tal forma que cuando nuestro coche avance esta spotlight se muestre apuntando hacia enfrente y en caso de que nuestro coche retroceda apunte hacia la parte trasera.

```
if (mainWindow.getLuces() < 0.5f)
{
    spotLights[1] = aux[1];
    spotLights[1].SetPos(glm::vec3(0.0f + cochePos.x, 2.5f + cochePos.y, -5.6f + cochePos.z));
}
else
{
    spotLights[1] = aux[0];
    spotLights[1].SetPos(glm::vec3(0.0f + cochePos.x, 3.395f + cochePos.y, -23.74f + cochePos.z));
}
shaderList[0].SetSpotLights(spotLights, spotLightCount);
```

En esta sección del código vemos la lógica que se maneja, para poder verlo de manera practica la luz trasera es de color rojo y la delantera de color azul.



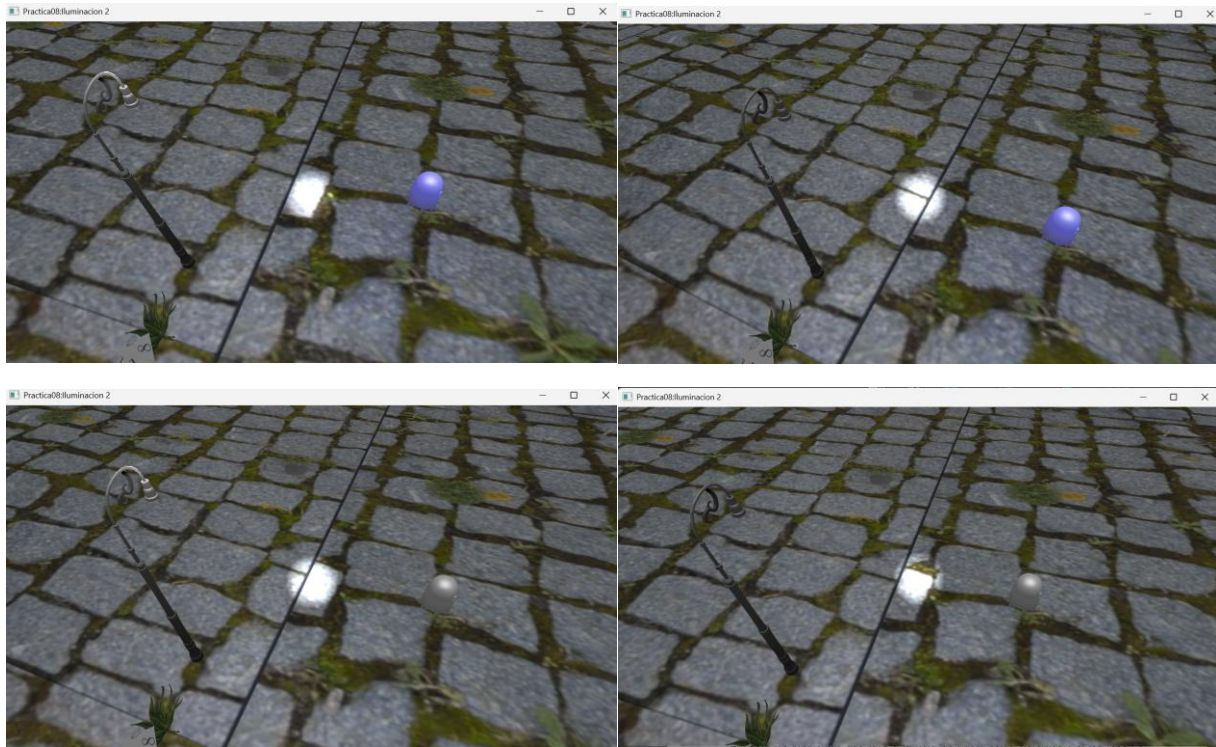
La asignación de teclas es sencilla, pero debemos tener cuidado al limitar el intervalo donde se trabaja, ya que una mala ejecución puede causar fenómenos extraños.

El ultimo ejercicio nos pide agregar una nueva luz puntual ligada a un modelo, en este caso un fantasma, y agregar la funcionalidad para que encienda y apague de manera independiente al faro utilizado en la práctica anterior. Las pointLights se encuentran ligadas a los modelos, es decir, si el modelo es trasladado la luz se trasladara de manera automática con este.

```
if (mainWindow.getLmp1() > 0.5f && mainWindow.getLmp2() > 0.5f) {
    shaderList[0].SetPointLights(pointLights, pointLightCount);
}
else if (mainWindow.getLmp1() > 0.5f && mainWindow.getLmp2() < 0.5f) {
    shaderList[0].SetPointLights(pointLights, pointLightCount - 1);
}
else if (mainWindow.getLmp1() < 0.5f && mainWindow.getLmp2() > 0.5f) {
    shaderList[0].SetPointLights(pointLights+1, 1);
}
else {
    shaderList[0].SetPointLights(pointLights, pointLightCount - 2);
}
```

En esta sección de código podemos observar la lógica de programación de las luces.

Son cuatro teclas (K,L,N,M) las que afectan las luces, dos para cada pointlight.



2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla.

No hubo problema durante la realización de la práctica.

3.- Conclusión:

Los ejercicios fueron realizados sin mayor problema, durante la clase vimos que solo pueden ser utilizadas un número máximo de luces, por lo que existen diversos métodos que hacen que el usuario tenga una experiencia donde parece haber más de luces que este número máximo, en estos ejercicios hicimos uso de estas dejando más en claro las futuras posibilidades en el aspecto de luces en el escenario.

4.- Bibliografía en formato APA

Ing. José Roque RG (septiembre 2025) Laboratorio de Computación Gráfica e Interacción Humano Computadora. Clase Facultad de Ingeniería UNAM.